

Logical Inference and Knowledge Representation

Maciej Piasecki, Halina Kwaśnicką

Definitions

- *Inference* – the process of generating (new) facts from (other) existing facts
- *Deduction* – a logically sound and systematic inference procedure
- *Induction* – reasoning from a part to the whole, which, in the context of learning, may involve agglomeration or classification of new information into larger entities

Inference

- Inference – a process of generating (new) facts from (other) existing facts
- Deduction – a logically (formally) sound (and possibly complete) method of inference ($((p \rightarrow q) \wedge p)$ is true then q is true)
- Abduction – inference about probable causes ($((p \rightarrow q) \wedge q)$ is true then p is true?)
- Induction – inference from a part to a whole that, in the context of learning, may include agglomeration or classification of new information to larger entities

Implication

$$p \rightarrow q$$

The term on the left (the antecedent) *implies* the term on the right (the consequent)

$$(p \rightarrow q) \equiv \neg p \cup q$$

The truth table (Implication)

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

The truth table (Modus Ponens)

p	q	$p \rightarrow q$	$(p \cap (p \rightarrow q))$
T	T	T	T
T	F	F	F
F	T	T	F
F	F	T	F

Controversy with implication

1. p and q together do not need to make sense,
IF $4+2=3$
THEN Dogs are smarter than people
3. A FALSE premise (hypothesis) implies any conclusion
4. Implication $p \rightarrow q$ does not mean
“ p causes q ”
5. A simple statement could be viewed as an
implication with no antecedents (q)

Deduction

- The deductive inference uses the implication operator:
- It is assumed that the $p \rightarrow q$ implication is true,
- From the truth of the implication and from the truth of the premise p (which may be a complex sentence) the truth of the conclusion q is deduced

True: $p \rightarrow q$

True: p

Infer: q is True

From a certain general (known) rule and fact (known) rule, we deduce a new fact (top-down inference) - from general to specific

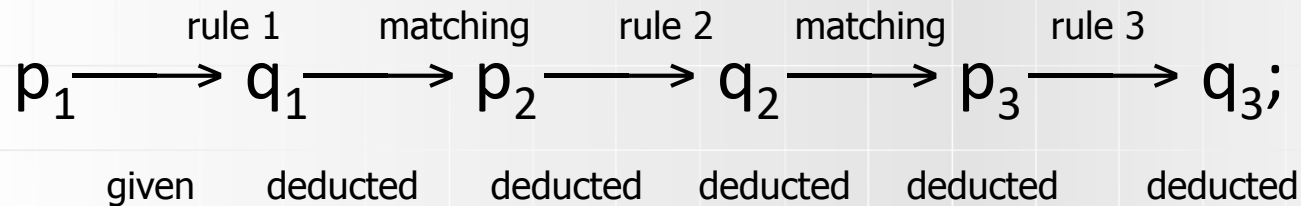
Definitions

- A **term** is: a constant, a variable, or an n -argument function
- A **function** is an n -tuple of terms prefixed by a function symbol or a name (called also *functor*) that satisfies the definition of a function
- A **predicate** is an n -tuple of terms, prefixed by a predicate symbol or name
- A less formal definition of **predicate**: a predicate is a parameterised proposition, that is, a proposition with variables.
- A **predicate function** on a set A is a function that maps elements of A into the set $\{\text{TRUE}, \text{FALSE}\}$
- Example: A – the set of integers, *odd* predicate is:

Deduction in propositional logic

- **implication 1:**
 - p_1 A college professor teaches in the summer
 - q_1 The professor can't do anything but teach
- **implication 2:**
 - p_2 The professor can't do anything but teach
 - q_2 The professor does not have time to do research
- **implication 3:**
 - p_3 The professor does not have time to do research
 - q_3 The professor is unhappy
- **Determine:** truth of statement q_3 , given the fact p_1 is true
- **Knowledge base:** $p_1 \rightarrow q_1; p_2 \rightarrow q_2; p_3 \rightarrow q_3;$
 $p_2 = q_1; q_2 = p_3;$
- **Knowledge base rewritten:** $p_1 \rightarrow q_1; q_1 \rightarrow q_2; q_2 \rightarrow q_3;$

Inference net for the three-rule deduction example



- matching – a symbolic matching approach, it ascertains *equality* of statement truth-values, it can be done by a procedure invoked by an inference mechanism

Forward Chaining paradigm

Inference net and rule firings:

1. $(p_1 \rightarrow q_1) \cap p_1 \text{ being TRUE} \rightarrow (q_1 \text{ is TRUE})$
2. $(q_1 \text{ is TRUE}) \text{ and } (q_1 = p_2) \text{ being TRUE yields } p_2 \text{ is TRUE}$
3. [Similar to step 1]
 $(p_2 \rightarrow q_2) \cap p_2 \text{ being TRUE} \rightarrow (q_2 \text{ is TRUE})$
4. [Similar to step 2]
 $(q_2 \text{ is TRUE}) \text{ and } (q_2 = p_3) \text{ being TRUE yields } p_3 \text{ is TRUE}$
5. [Similar to step 1 and 3]
 $(p_3 \rightarrow q_3) \cap p_3 \text{ being TRUE} \rightarrow (q_3 \text{ is TRUE})$

Forward Chaining

1. Reasoning starts with at least one fact.
2. For each condition Inference Engine looks for rules in the Knowledge Base which in the part IF corresponds to the condition (conditions)
3. If the antecedent is true, the conclusion is produced as true (deducted facts are added to the system)
4. For each added fact Inference Engine returns to the point 2
5. Inference stops when all conclusions (facts) are analysed. Deducted facts are considered as the system output

Example

Knowledge Base:

10. IF INTEREST=FALL

THEN STOCK=RISE

20. IF INTEREST=RISE

THEN STOCK=FALL

30. IF DOLLAR=FALL

THEN INTERES=RISE

40. IF DOLLAR=RISE

THEN INTEREST=FALL

50. IF FEDIN=FALL AND

FEDMON=ADD

THEN INTEREST=RISE

Variable name	Meaning
INTEREST	Interest rate direction
DOLLAR	Dollar exchange rate
FEDIN	Federal reserve interest rate goals
FEDMON	Federal reserve money circulation goal (i.e., add or remove reserves from the banking system)
STOCK	Stock market direction

Processing the Knowledge Base

R10. 1. **INTEREST**
2.
3.
4.
R20. 5. **INTEREST**
6.
7.
8.
R30. 9. **DOLLAR**
10.
11.
12.
R40. 13. **DOLLAR**
14.
15.
16.
R50. 17. **FEDIN**
18. **FEDMON**
19.
20.

Clause variable list

INTEREST **FALL**
DOLLAR **RISE**
FEDIN ...
FEDMON ...

Variable list

Rule no.	Clause no.
----------	------------

Clause variable pointer

DOLLAR
INTEREST
STOCK

Conclusion variable queue

Knowledge Base:

10. IF INTEREST=FALL THEN STOCK=RISE
20. IF INTEREST=RISE THEN STOCK=FALL
30. IF DOLLAR=FALL THEN INTERES=RISE
40. IF DOLLAR=RISE THEN INTEREST=FALL
50. IF FEDIN=FALL AND FEDMON=ADD
THEN INTEREST=RISE

Initial fact: **DOLLAR=FALL**

Inference: **INTEREST=RISE**

Inference: **STOCK=FALL**

Backward Chaining

- Knowledge Base:

10. **IF** DEGREE=NO **THEN** POSITION=NO

20. **IF** DEGREE=YES **THEN** QUALIFY=YES

30. **IF** DEGREE=YES **AND** DISCOVERY=YES
THEN POSITION=RESEARCH

40. **IF** QUALIFY=YES **AND** GRADE < 3.5 **AND** EXPERIENCE ≥ 2
THEN POSITION=SERVICE ENGINEER

50. **IF** QUALIFY=YES **AND** GRADE < 3.5 **AND** EXPERIENCE < 2
THEN POSITION=NO

60. **IF** QUALIFY=YES **AND** GRADE ≥ 3.5
THEN POSITION=PRODUCT ENGINEER

Inference - example

10. POSITION
20. QUALIFY
30. POSITION
40. POSITION
50. POSITION
60. POSITION

conclusion list

DEGREE **YES** value
DISCOVERY value
EXPERIENCE value
GRADE value

variable list

QUALIFY: YES

GRADE ?: 4

Hypothesis: POSITION=PRODUCT
ENGINEER

10:
1.DEGREE
2.

3.
4.

20:
1.DEGREE
2.

3.
4.

30:
1.DEGREE
2.DISCOVERY

3.
4.

40:
1.QUALIFY
2. GRADE
3.EXPERIENCE

4.

50:
1.QUALIFY
2.GRADE
3.EXPERIENCE

4.

60:
1.QUALIFY
2.GRADE

3.
4.

Clause variable list

10. IF DEGREE=NO THEN POSITION=NO
20. IF DEGREE=YES THEN QUALIFY=YES
30. IF DEGREE=YES AND DISCOVERY=YES
THEN POSITION=RESEARCH
40. IF QUALIFY=YES AND GRADE < 3.5 AND
EXPERIENCE ≥ 2
THEN POSITION=SERVICE ENGINEER
50. IF QUALIFY=YES AND GRADE < 3.5 AND
EXPERIENCE < 2
THEN POSITION=NO
60. IF QUALIFY=YES AND GRADE ≥ 3.5
THEN POSITION=PRODUCT ENGINEER

20	1
60	2
Rule no.	Clause no.

Conclusion stack

Knowing: DEGREE=YES

Predicate-based representation

- We want to represent the statement:
The circuit breaker in line 4 is open.
- In predicate calculus we can write:
 - *is-open (circuit-breaker, line-4)*
 - *circuit-breaker (line-4, is-open)*
 - *line-4 (circuit-breaker, is-open)*
- All are valid representations

Predicate-based representation

- General two-place relations:
 - *is-open* (*What*, *Where*)
 - *circuit-breaker* (*Where*, *Status*)
 - *line-4* (*Protection*, *Status*)
- Example: a universe matching the predicates
 - *What* = {*circuit-breaker*, *disconnect-switch*, *line-itself*}
 - *Where* = {*line-1*, *line-2*, *line-3*, *line-4*}
 - *Status* = {*open*, *closed*, *inoperable*}
 - *Protection* = {*relay*, *circuit-breaker*, *recloser*}
- Often, a predicate is inspired by a verb

Resolution (Proof by Refutation)

- Well formed formula (**wff**) is a a predicate or statement (with negation, conjunction, disjunction, implication, quantification)
- Clauses:
 - are a subset of logical sentences
 - are built using predicates and logical connectives
 - are allowed to contain variables that are assumed universally quantified
- A clause can be proven to be true in the context of a set of clauses known to be true, by appending the logical NOT of this clause to the set and seeking a contradiction

Pairwise resolution

- $p \rightarrow q$ is replaced with $((\neg p) \vee q)$
- $p \equiv q$ is replaced with $((\neg p) \vee q) \wedge (p \vee \neg q)$
- Resolution (simplified)

$$\neg p \vee q$$
$$p \vee r$$

$$\vdash q \vee r$$

Resolution - example

D1: goal: (q2=?)

1. p1
2. p1 → q1
3. q1 → q2

(1) $p \rightarrow q$ is replaced with $((\neg p) \cup q)$

D2:

1. p1
2. $\neg p1 \vee q1$
3. $\neg q1 \vee q2$
4. $\neg q2$

(2) Resolution of (2) and (3), that is:

$$\begin{array}{r} q1 \vee \neg p1 \\ \neg q1 \vee q2 \\ \hline \neg p1 \vee q2 \end{array}$$

D3:

1. p1
2. $\neg p1 \vee q1$
3. $\neg q1 \vee q2$
4. $\neg q2$
5. $\neg p1 \vee q2$

D4:

1. p1
2. $\neg p1 \vee q1$
3. $\neg q1 \vee q2$
4. $\neg q2$
5. $\neg p1 \vee q2$
6. $\neg p1$

(3) Resolution of (4) and (5), that is:

$$\begin{array}{r} \neg q2 \\ \underline{q2 \vee p1} \\ p1 \end{array} \quad \begin{array}{r} \neg q2 \\ \underline{q2 \cup \neg p1} \\ \neg p1 \end{array}$$

Simple inference and knowledge representation: Logical circuits

1. If two terminals are connected, then they have the same signal:

$$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2) .$$

2. The signal at every terminal is either 1 or 0:

$$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0 .$$

3. Connected is commutative:

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1) .$$

4. There are four types of gates:

$$\forall g \text{ Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT} .$$

5. An AND gate's output is 0 if and only if any of its inputs is 0:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{AND} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0 .$$

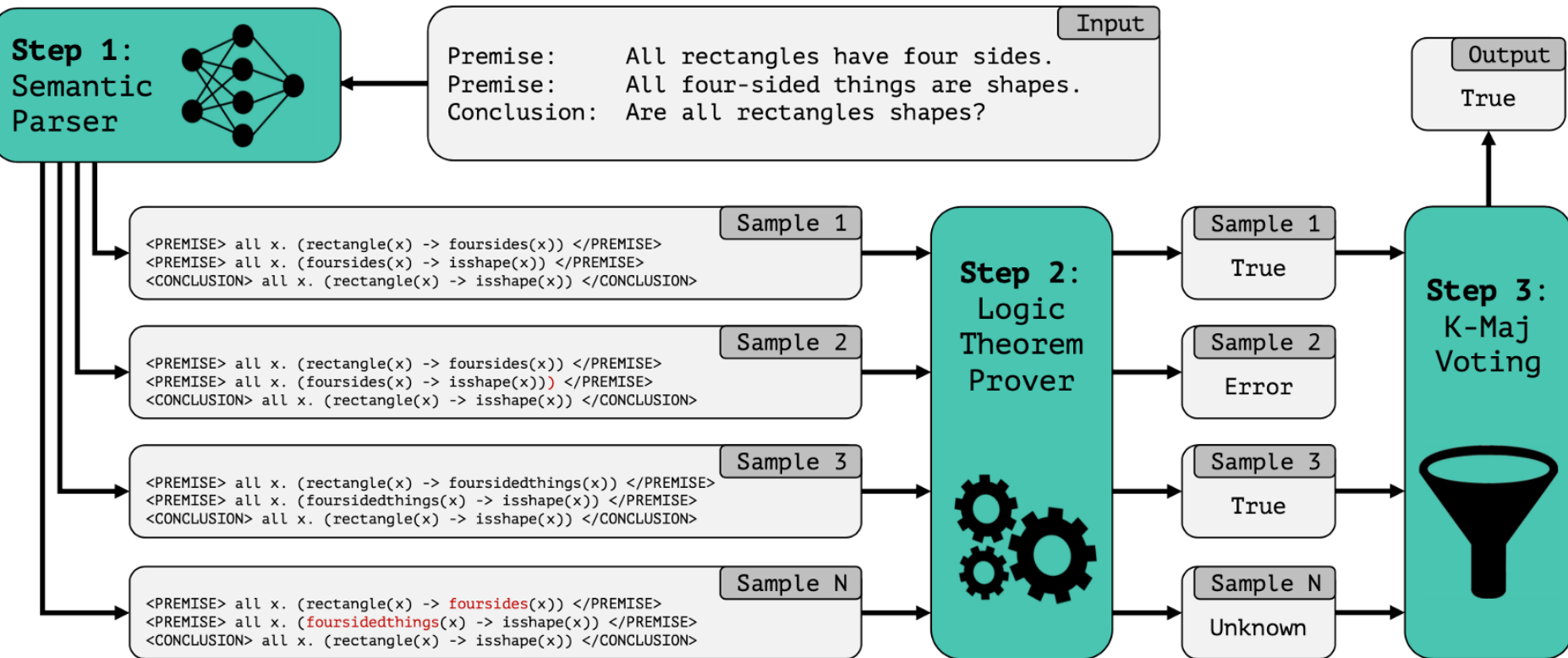
6. An OR gate's output is 1 if and only if any of its inputs is 1:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{OR} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 1 .$$

Logic

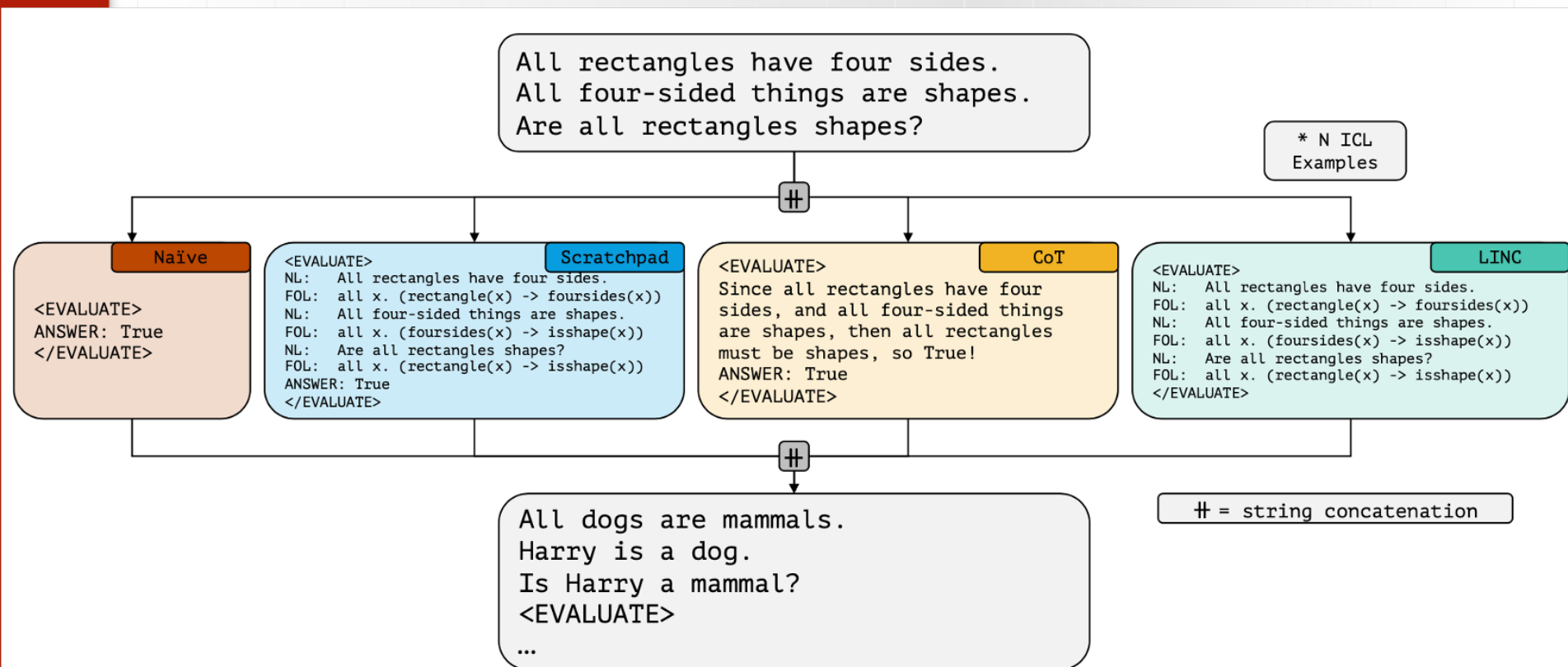
- Classical logic is *monotonic*: anything that follows from a set of facts, also follows from an expanded set of facts
- Everyday experience indicates that observation of new facts often causes humans to revise a set of the previous facts (or “beliefs”) – *nonmonotonic logic*
- The concept of time is often significant to human reasoning – classical logic may be extended to include *temporal logic*
- In reasoning with uncertainty, classical logic may be modified to embody *fuzzy logic*, *probabilistic logic*, *multivalued logic*

LINC – theorem prover in LLM



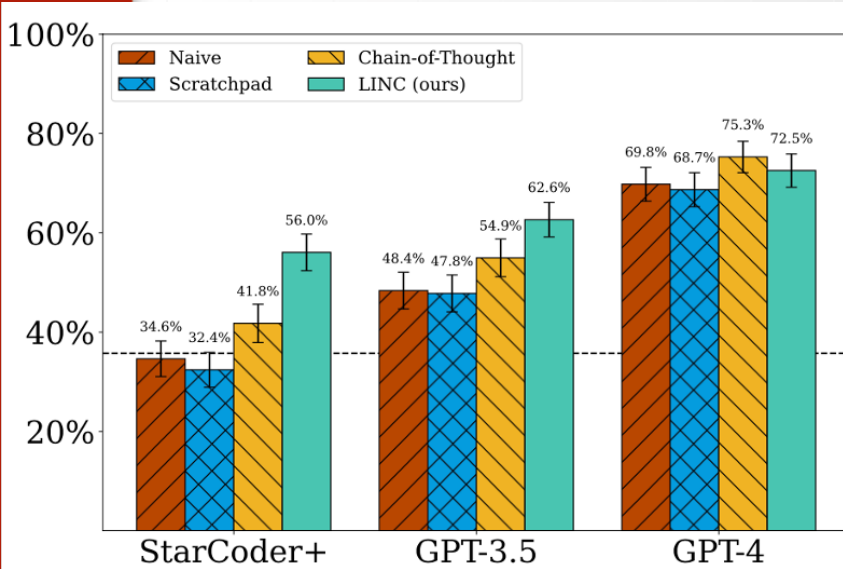
Prompt → logical sentence → inference → voting

LINC – comparison

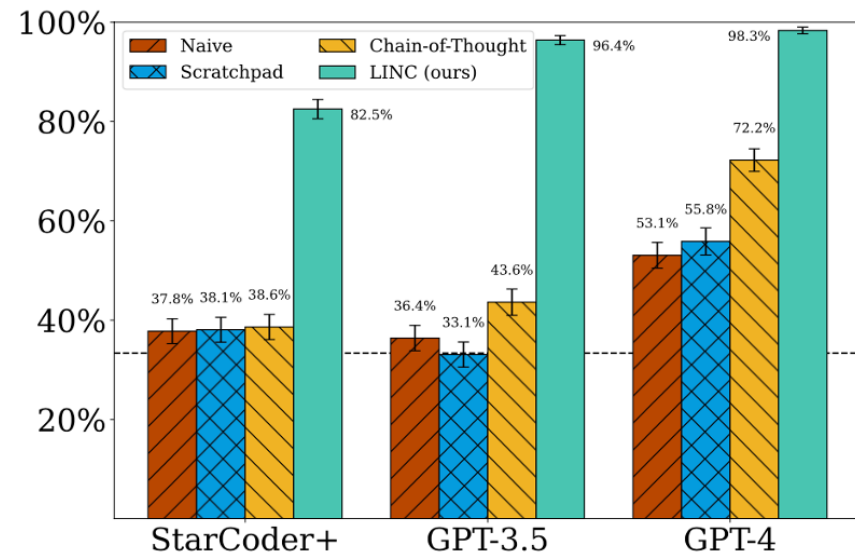


Naïve – contextual prompt, Scratchpad – generation of a formula, without inference, CoT = Chain of Thought, LINC – the use of the theorem prover

LINC – comparison



(a) FOLIO.



(b) ProofWriter.

Theo X. Olausson et al. (2023) LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, ACL pp. 5153–5176.

Predicates and variables

- Predicates with a single argument are often used to represent *unary relations*, which is commonly referred to as **properties**:
 - *may_move* (*Object*)
 - *next_to* (*Region1*, *Region2*)
 - *on_top* (*Support*, *Object*)

Subjects to be represented

- **Objects:** Jon, White House, Azor
- **Properties** of objects: Mary is blond
- **Relations** between objects: Mary likes dogs
- **Classes** of objects: people, dogs, animals
- **Types** of events: accident, meeting
- **Events:** Mary lost her handbag in the office
- **Relations** between events: Mary has fallen asleep watching TV because she was tired
- **Effects** of events: Mary lost handbag, and now she does not have her passport
- **Statements:** If someone lost the passport he has to receive the new one. All students are intelligent.

Properties

- Possibility to record such information
- Cope with difficulties caused by, e.g., conditional information or contradictory information
 - some sentences can be true only in specific situations (conditional information)
 - some theories contain exceptions (contradictory information)

Additional:

- a system can contain knowledge about the knowledge stored in the system – meta-knowledge
- in a system having more than one inference method, meta-knowledge allows choosing suitable inference strategy for a given problem

$\langle O, A, V \rangle$ triples

- Sentence **an attribute R of object x has value y** can be written as:
 - function f , which returns y : $y = f(x, R)$
 - relation R : $\langle R, x, y \rangle$ (R, x - finite sets), often we wrote:
 $\langle \text{Object, Attribute, Value} \rangle$: $\langle \text{leaf, colour, green} \rangle$
- The relations between symbolic values are important knowledge component
- Properties
 - 1st order unary relations (with one argument)
 - Attributes that are general features of objects (e.g. size, shape, colour, or value)
- Properties define particular state of object in a given moment:
 $\langle \text{animal, breathe} \rangle$

N-ary relations

- Binary relation – relations between two objects
- N-ary relations can be presented by using nested binary relations:
 - a triple relation between A, B and C is a Cartesian product of two sets $(A \times B) \times C$ described as ordered triple $((a,b),c)$
 - relation of 4 arguments: A,B,C,D subset of $((A \times B) \times C) \times D$
 - an ordered four $((a,b),c)d$
- Binary and n-ary relations can be presented in a form of a relation table

Semantic network

- Graphical representation of knowledge
- Labelled digraph, in which nodes and edges have labels
 - nodes usually represent physical *objects* (animals, things) or abstract *concepts*, attributes - actions, situations, concepts, attributes (features of objects)
 - edges usually join nodes representing objects with nodes representing attributes

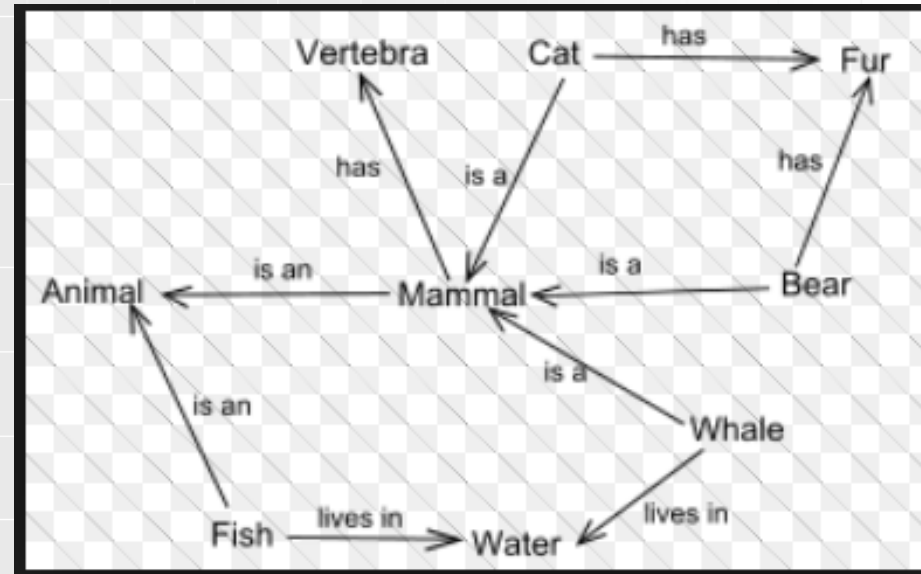
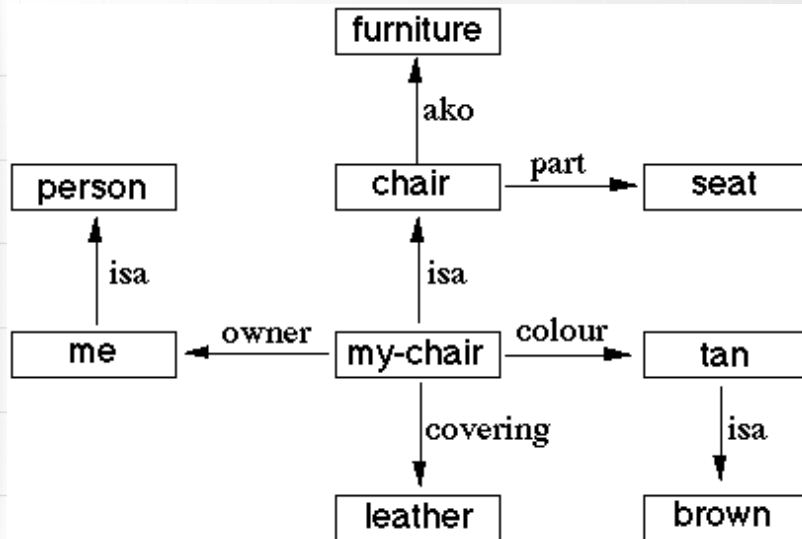
Definition

- Semantic network G is ordered triple $G=(V,S,u)$, where:
 - V is a set of nodes,
 - S – a set of edges between nodes ($S\subseteq V\times V$),
 - u is a function defining a property assigned to a given edge,
 $u: S\rightarrow W\times N$ where W – a set of properties, N – numbers,
weights assigned to particular edges (connections)
- Often weights of all connections are equal to one (set N has one element, is excluded in definition)

Relations

- Semantic networks represent different relations
 - *has*
 - *has_part*
 - *is_part*
 - *uses* etc.
- Among them two very important relations which allow representing inheritance (subsumption):
 - relation *instance*
 - relation *is_a*

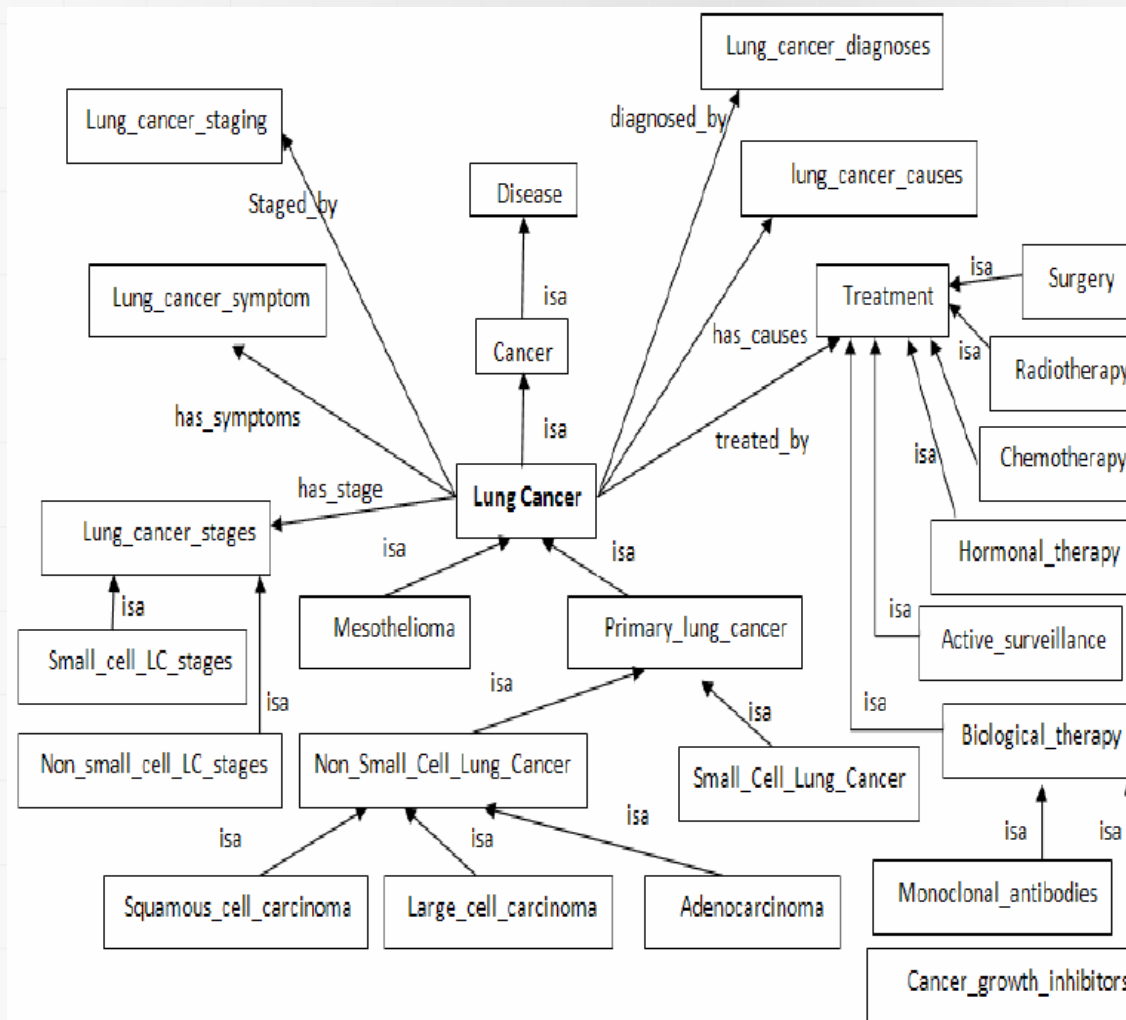
Examples



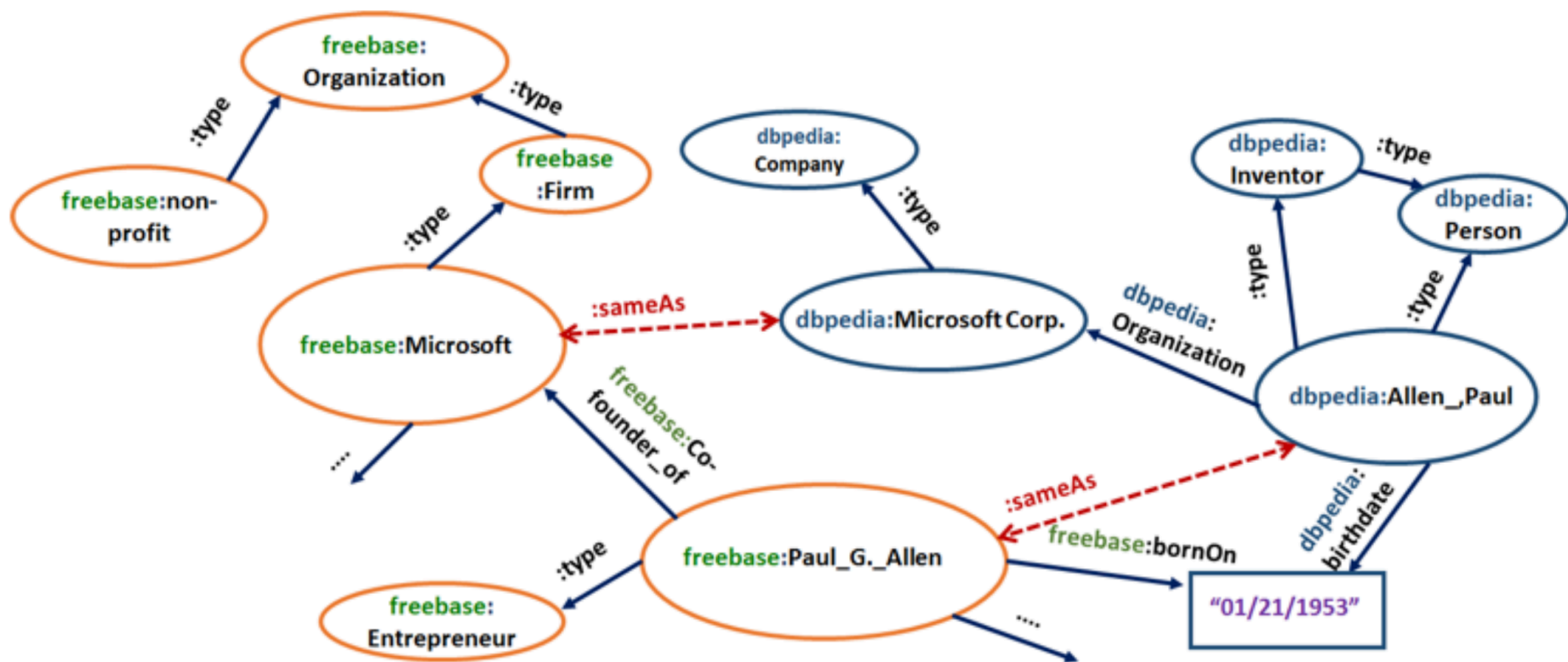
<http://www.cse.unsw.edu.au/~billw/cs9414/notes/kr/frames/frames.html>

https://upload.wikimedia.org/wikipedia/commons/6/67/Semantic_Net.svg

Example



Examples: freebase and DBpedia



https://www.researchgate.net/figure/Fragments-of-real-world-RDF-graphs-DBpedia-and-Freebase-with-two-examples-of-Entity_fig1_318408896